

Bentix Architecture

Overview

Bentix is an operational management platform for teams and construction works. The application brings together a single product surface for:

- user authentication and session management
- people management and access identities
- clients and works
- daily planning and work assignments
- time entry, submission, and approval
- materials, notifications, and technical support tools

Technology stack

- `Next.js 16` with `App Router`
- `React 19`
- `Node.js 22` in Docker
- `Prisma 7` with `@prisma/adaptor-mariadb`
- `MariaDB 11.4`
- `Nginx` as reverse proxy
- `Cloudflare` for DNS and edge proxying
- `Swagger UI` / OpenAPI for API documentation
- `node:test` for critical automated tests

Architecture principles

- a single monorepo for frontend, HTTP API, and data access
- thin `app/api/*` routes with no meaningful business logic
- clear separation between `controllers`, `services`, and `lib`
- centralized public environment configuration in `config/`
- HTTP-only cookie authentication with signed sessions
- MySQL/MariaDB as the main operational data source
- preparation for a future real frontend/backend split without breaking current contracts

General Architecture

Main logical diagram:

```
Browser
|
v
Web (Next.js)
|
REST API
|
MariaDB
```

Role of each layer

Browser

The browser runs the React interface delivered by Next.js, stores the session cookie, and performs `fetch` calls to the API with `credentials: include`.

Web (Next.js)

Next.js has three responsibilities:

- serve the application pages and components
- expose the API under `app/api/*`
- execute `proxy.js` for access guards, HTTPS redirects in production, and API preflight/CORS handling

In practice, the app remains monolithic. Even when the frontend and API are published under different subdomains, the code still lives in the same repository and the same Docker image is reused.

REST API

The API uses the following internal chain:

```
app/api/*/route.js
|
v
server/controllers/*
|
v
server/services/*
|
v
lib/* + lib/db/*
```

The `route.js` files receive the HTTP request, the `controllers` handle the HTTP boundary, the `services` centralize application rules, and `lib/db/*` talks directly to Prisma.

MariaDB

MariaDB is the target persistent database. Access is done through Prisma and centralized in `lib/prisma.js` and `lib/prisma-adapter.js`.

Current deployment diagram

```
Browser
|
v
Cloudflare
|
v
Nginx
|
+-- dev.bentixapp.com  ----> web / app
|-- api-*.bentixapp.com ----> api / app
                             |
                             v
                             Prisma
                             |
                             v
                             MariaDB
```

Environments

The supported architecture includes only:

- LOCAL
- DEV
- PROD

Environment summary

Environment	Frontend	API	Database	Execution
LOCAL	<code>http://localhost:3000</code>	same origin under <code>/api/*</code>	local MySQL/MariaDB, typically <code>bentix_local</code>	<code>npm run dev</code>
DEV	<code>https://dev.bentixapp.com</code>	<code>https://api-dev.bentixapp.com</code>	Docker MariaDB <code>bentix_dev</code>	<code>docker compose</code>
PROD	<code>https://bentixapp.com</code>	<code>https://api.bentixapp.com</code>	Docker MariaDB <code>bentix_prod</code>	<code>docker compose</code>

LOCAL

- the frontend and API run in the same Next.js process
- the browser uses relative URLs such as `/api/*`
- there is no subdomain split
- the cookie can remain host-only, without `Domain`
- the database is configured through `DATABASE_URL`

DEV

- the frontend is published at `dev.bentixapp.com`
- the API is published at `api-dev.bentixapp.com`
- `docker-compose` creates three physical services: `web`, `api`, and `db`
- `web` and `api` use the same Next.js application, but with different publication roles
- for cross-subdomain session sharing, the cookie should use `SESSION_COOKIE_DOMAIN=.bentixapp.com`

PROD

- the frontend is published at `bentixapp.com`
- the API is published at `api.bentixapp.com`
- today the production `docker-compose` has two physical services: `app` and `db`
- logically there are `web` and `api` roles, but both are exposed through the same `app` service behind Nginx
- this keeps the current architecture simple without changing application logic

Environment configuration

The public API URL configuration lives in:

- `config/app.local.js`
- `config/app.dev.js`

- `config/app.prod.js`
- `config/app.public.js`

Rules:

- `NEXT_PUBLIC_APP_ENV` accepts only `local`, `dev`, or `prod`
- `NEXT_PUBLIC_API_BASE_URL`, when defined, overrides the profile URL
- without an override, the app uses the URL defined in `config/app.<environment>.js`

Application Structure

```
frontend/  
app/  
server/  
lib/  
config/  
infra/  
tests/
```

`frontend/`

Client-side API consumption layer for React components.

- `frontend/api/api-client.js` resolves the base URL and executes `fetch`
- `frontend/controllers/*` wraps API calls and avoids spreading transport details across the UI

`app/`

Next.js App Router layer.

- pages and layouts
- React server/client components
- HTTP routes under `app/api/*`
- Swagger UI at `/developer/api-docs`

`server/`

HTTP and application layer.

- `server/controllers/*`: request parsing, session, authorization, and error mapping
- `server/services/*`: application logic and orchestration
- `server/responses/*`: HTTP response helpers
- `server/errors/*`: typed errors
- `server/docs/*`: OpenAPI served by the application itself

lib/

Cross-cutting domain and internal infrastructure layer.

- authentication, cookies, and session
- permissions, roles, and access profiles
- entity-level data access
- reusable business helpers
- legacy compatibility and a few technical fallbacks

Inside `lib/` :

- `lib/db/*` contains entity-specific data access
- `lib/prisma.js` and `lib/prisma-adapter.js` initialize Prisma
- `lib/data-source.js` chooses between `mysql` and `json` fallback

config/

Public application configuration by environment.

- selects the API base URL consumed by the frontend
- isolates environment configuration from business code

infra/

Versioned infrastructure.

- `infra/environments/*` : `docker-compose` and `.env.example`
- `infra/docker/app/Dockerfile` : application image build and runtime
- `infra/nginx/*` : reverse proxy per environment
- `infra/scripts/*` : operational utilities

tests/

Critical automated tests.

- authentication flow
- CORS
- public environment configuration
- session cookie options
- critical planning and assignment flows

Authentication Flow

Complete flow:

```
Browser
  |
  v
GET /login
  |
  v
GET /api/auth/payload-key
  |
  v
Browser encrypts credentials
  |
  v
POST /api/auth/login
  |
  v
Cookie bentix_session
  |
  v
GET /api/auth/session and protected routes
  |
  v
GET /api/auth/logout
  |
  v
Cookie expired
```

Step by step

1. The user opens `/login`.
2. The frontend requests the public key from `/api/auth/payload-key`.
3. The browser encrypts the login payload with AES-GCM and encrypts the ephemeral key with RSA-OAEP.
4. The frontend sends `POST /api/auth/login` with `protectedPayload`.
5. `server/controllers/auth-login-controller.js` reads the protected body.
6. `lib/login-transport.js` decrypts the payload.
7. `server/services/auth-login-service.js` validates username/password, checks lockouts, updates audit data, and creates the session token.
8. `lib/auth.js` signs the session with `AUTH_SECRET` and returns an HMAC token.
9. The response sets the `bentix_session` cookie with `HttpOnly`, `Path=/`, and environment-specific `SameSite / Domain` behavior.
10. In later calls, the browser automatically sends the cookie because `frontend/api/api-client.js` uses `credentials: include`.
11. `proxy.js` and `lib/server-session.js` validate the session before allowing access to protected pages and endpoints.

12. In `/api/auth/logout`, the application expires the same cookie using the same base options.

Session rehydration

The application does not rely only on the raw cookie. After validating the signature, `lib/server-session.js` loads the user and access state again from the active data source.

This allows the app to:

- invalidate disabled accounts
- recalculate permissions and access profiles
- refresh `workIds` and the operational identity state

Session cookie

Current behavior:

- `HttpOnly=true`
- `Path=/`
- `SameSite=Lax` on local host-only mode without `SESSION_COOKIE_DOMAIN`
- `SameSite=None` when `SESSION_COOKIE_DOMAIN` exists
- `Secure=true` in production and whenever `SESSION_COOKIE_DOMAIN` exists

Expected example for DEV/PROD with frontend and API on different subdomains:

```
Domain=.bentixapp.com; SameSite=None; Secure; HttpOnly; Path=/
```

REST API Call Flow

Requested diagram:


```
Browser
|
v
Web
|
v
api-client
|
v
REST API
|
v
Service
|
v
Prisma
|
v
MariaDB
```

Step-by-step explanation

1. Browser

The user interacts with the page or component.

2. Web

The Next.js frontend renders the UI and triggers a user action. In client-side pages, this happens inside React components or `frontend/controllers/*`.

3. api-client

`frontend/api/api-client.js`:

- validates the requested path
- resolves the base URL through `config/app.public.js`
- uses `NEXT_PUBLIC_APP_ENV` and `NEXT_PUBLIC_API_BASE_URL`
- executes `fetch(..., { credentials: 'include' })`

In `LOCAL`, the final URL is relative, for example `/api/people`.

In `DEV`, the final URL is typically `https://api-dev.bentixapp.com/api/people`.

In `PROD`, the final URL is typically `https://api.bentixapp.com/api/people`.

4. REST API

The request enters a `route.js` under `app/api/*`. The route:

- calls a controller

- converts the result into `NextResponse`
- converts typed errors into HTTP status codes

Before the request reaches the controller, `proxy.js` may:

- answer preflight `OPTIONS`
- apply CORS headers for cross-origin calls
- reject requests without session or permission

5. Service

The controller delegates application logic to `server/services/*`.

Typical responsibilities:

- validate session
- verify permissions
- validate input
- orchestrate operations

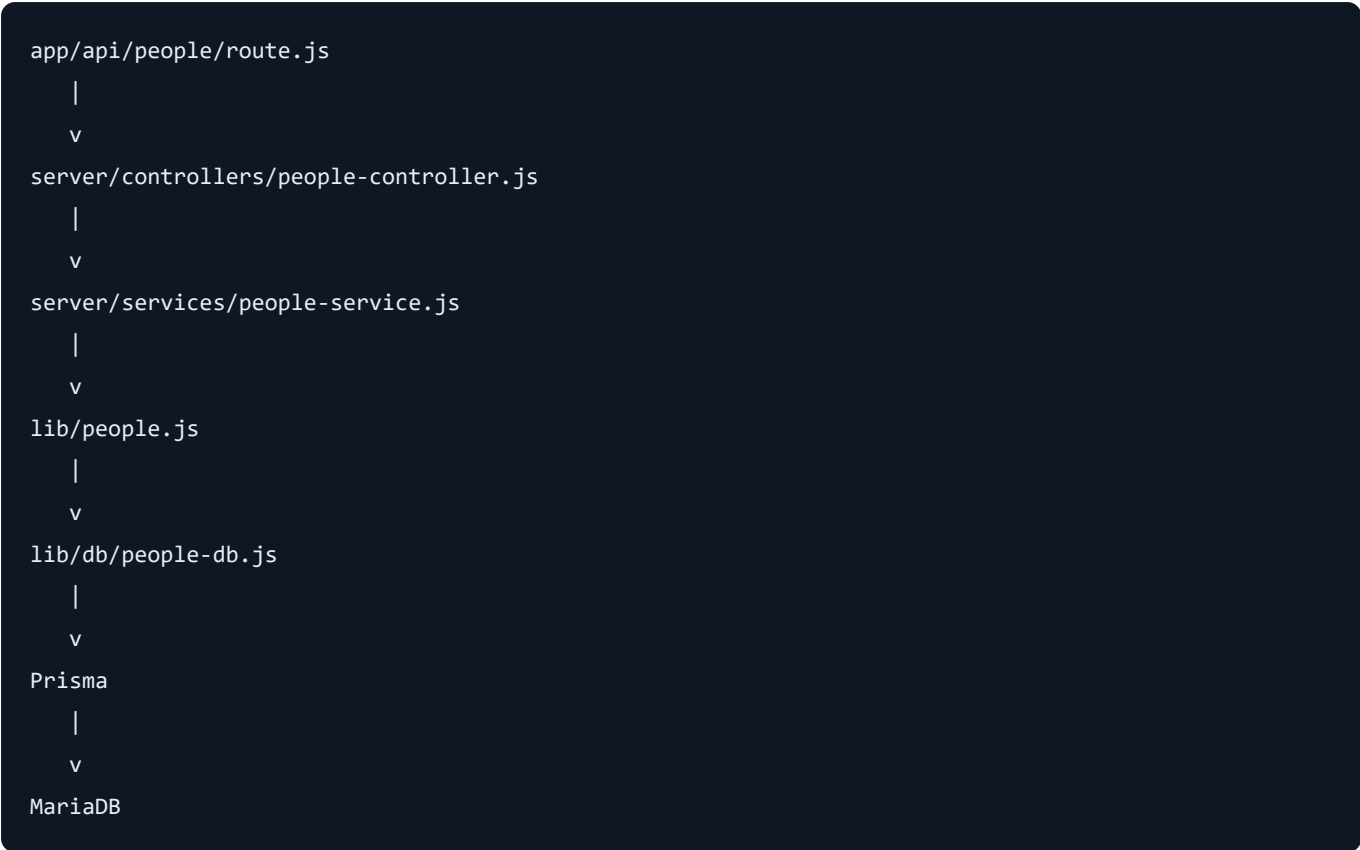
6. Prisma

The `services` use `lib/*` and `lib/db/*`, which in turn use Prisma to talk to MariaDB.

7. MariaDB

MariaDB persists the application state: people, users, works, assignments, events, permissions, and the remaining entities.

Real example: GET /api/people



Configuration

Main variables

Variable	Role	Where it is used	Primary read time
NEXT_PUBLIC_APP_ENV	selects the public profile local/dev/prod	config/app.public.js , frontend bundle	build time
NEXT_PUBLIC_API_BASE_URL	explicit API base URL override	config/app.public.js , frontend bundle	build time
SESSION_COOKIE_DOMAIN	enables cookie sharing across subdomains	lib/auth.js	runtime
DATABASE_URL	Prisma/MariaDB connection	lib/prisma.js , lib/prisma- adapter.js	runtime
AUTH_SECRET	HMAC signing secret for the session	lib/auth.js	runtime

NEXT_PUBLIC_APP_ENV

- supported values: `local`, `dev`, `prod`
- selects `config/app.local.js`, `config/app.dev.js`, or `config/app.prod.js`
- is embedded into the application during build

NEXT_PUBLIC_API_BASE_URL

- is optional
- when defined, it takes priority over the profile URL
- is useful to force an alternative API without changing code

SESSION_COOKIE_DOMAIN

- empty in `LOCAL`, so the cookie remains host-only
- `.bentixapp.com` in `DEV` and typically also in `PROD`, to share session between `web` and `api`

DATABASE_URL

- defines the real database used by the application at runtime
- a dummy `DATABASE_URL` also exists during Docker build so `prisma generate` and `next build` can run
- that dummy value must not be confused with the real production connection

AUTH_SECRET

- secret used to sign and validate session content
- mandatory in production
- without it, the application cannot validate session cookies securely

Build time vs runtime

Build time

- `NEXT_PUBLIC_APP_ENV`
- `NEXT_PUBLIC_API_BASE_URL`

Runtime

- `DATABASE_URL`
- `AUTH_SECRET`
- `SESSION_COOKIE_DOMAIN`
- `LOGIN_PUBLIC_KEY_PEM`
- `LOGIN_PRIVATE_KEY_PEM`

Important note:

- in the current Docker setup, `NEXT_PUBLIC_*` enters as `ARG` and `ENV` before `next build`
- for browser behavior, the public configuration should therefore be treated as build-time configuration

Docker

Application image

The Dockerfile in `infra/docker/app/Dockerfile` :

- uses `node:22-bookworm-slim`
- installs dependencies
- runs `npm run db:generate`
- runs `npm run build`
- copies `.next` , code, and the required artifacts into the final image

DEV

Current physical topology:

```
web  -> frontend published at dev.bentixapp.com
api  -> same Next.js app published at api-dev.bentixapp.com
db   -> MariaDB for the dev environment
```

Notes:

- `web` and `api` use the same image
- `web` receives `NEXT_PUBLIC_APP_ENV=dev`
- `api` receives `NEXT_PUBLIC_APP_ENV=local` so it does not point itself to another public API

How to start:

```
cd infra/environments/dev
cp .env.example .env
docker compose up -d --build
```

PROD

Desired logical topology:

```
web  -> bentixapp.com
api  -> api.bentixapp.com
db   -> production MariaDB
```

Current physical topology in the repository:

```
app  -> serves both web and api roles
db   -> production MariaDB
```

This means:

- `bentixapp.com` and `api.bentixapp.com` both point to the same `app` service
- the production split is currently logical and hostname-based, not based on two separate containers

How to start:

```
cd infra/environments/production
cp .env.example .env
docker compose up -d --build
```

Nginx

DEV

`infra/nginx/dev.conf.example` publishes:

- `dev.bentixapp.com` -> `127.0.0.1:3100`
- `api-dev.bentixapp.com` -> `127.0.0.1:3101`

PROD

`infra/nginx/production.conf.example` publishes:

- `bentixapp.com` -> `127.0.0.1:3300`
- `api.bentixapp.com` -> `127.0.0.1:3300`

Nginx role

- terminates TLS
- publishes distinct hostnames
- routes traffic to the correct Docker services
- preserves headers such as `Host`, `X-Forwarded-For`, and `X-Forwarded-Proto`

Cloudflare

Minimum required DNS records:

Name	Suggested type	Target	Use
@	A or CNAME	VPS IP/hostname	PROD frontend
api	A or CNAME	VPS IP/hostname	PROD API
dev	A or CNAME	VPS IP/hostname	DEV frontend
api-dev	A or CNAME	VPS IP/hostname	DEV API

Practical recommendations:

- enable Cloudflare proxy when the environment is already public
- use TLS mode `Full (strict)` when the origin has a valid certificate
- ensure the hostnames published in Nginx also exist in DNS

Summary flow:



Deployment Flow

Reference manual flow:

```
Developer
|
v
git commit
|
v
git push
|
v
git pull on server
|
v
docker compose build
|
v
docker compose up
```

Current DEV equivalent

The repository already includes `/.github/workflows/deploy-dev.yml`, which automates DEV deployment:


```
Developer
|
v
push to the dev branch
|
v
GitHub Actions
|
v
npm ci
|
v
npm run test:critical
|
v
npm run build
|
v
SSH to the server
|
v
git fetch / reset
|
v
docker compose up -d --build
```

Project Conventions

How to add pages

1. Create the page in `app/<route>/page.js`.
2. If the page requires session, use `getSession()` and redirect when needed.
3. For client-side components, call `frontend/controllers/*` instead of doing ad hoc `fetch` calls throughout the UI.

How to add controllers

`server/controllers/*` should:

- read params, query, and body
- obtain session when needed
- transform protected requests with `readProtectedRequestJson`
- call exactly one or a few services
- return `jsonResponse(...)` or an equivalent response object

- avoid talking directly to Prisma

How to add services

`server/services/*` should:

- validate business rules
- verify permissions
- orchestrate calls to `lib/*`
- throw `HttpError` when the HTTP response needs an explicit status

How to add REST endpoints

1. Create `app/api/<route>/route.js`.
2. Keep the route thin, using `toNextResponse` and `toNextErrorResponse`.
3. Create or reuse the controller in `server/controllers/*`.
4. Create or reuse the service in `server/services/*`.
5. Update `server/docs/openapi-phase1.js` when the endpoint is public or documented.

How to add frontend API calls

1. Create a function in `frontend/controllers/<domain>-controller.js`.
2. Use `apiFetch` or `apiFetchJson`.
3. Always leave URL resolution to `frontend/api/api-client.js`.

How to add permissions

1. Declare the permission key in `lib/permissions.js`.
2. Add the key to the proper profiles in `ACCESS_PROFILE_PERMISSION_KEYS`.
3. Apply the rule in the service with `hasPermission(...)` or `requireSessionPermissionService(...)`.
4. If the new route/page has path-based guards, update the mapping in `lib/auth.js`.

HTTP boundary convention

The expected API pattern is:

```
route.js -> controller -> service -> lib -> lib/db -> Prisma
```

Any deviation from this pattern should be exceptional and justified.

Roadmap

Relevant remaining points before a more robust production operation:

- complete operational runbooks and documentation

- validate backup and restore end to end
- improve observability, logs, and alerts
- consolidate secret management outside local files
- automate rollout and rollback for production as well
- reduce remaining legacy file-based dependencies where they still exist
- keep the future backend split as an architectural evolution, not an immediate requirement

Practical roadmap reading

The current architecture already supports `LOCAL`, `DEV`, and `PROD`, but operational maturity is still evolving. The main gap is not the existence of pages or endpoints, but rather:

- operations automation
- monitoring
- secret discipline
- recovery routines

In summary:

```
Functional architecture: ready
Operational maturity: still evolving
Frontend/backend split: prepared, but not yet executed
```